TALLER DE OBSERVABILIDAD

Spring Boot · Prometheus · Loki · Grafana

Juan Carlos Tobón Montoya 160004937

EL PLAN: 20 MINUTOS

- Minutos 0 a 6: abre el Codespace, levanta el entorno y genera tráfico con k6.
- Minutos 6 a 16: tres misiones. La cola de latencia, cazar el N+1 y de la alerta al log.
- Minutos 16 a 20: reto. Crea una alerta de p99 mayor a 1 s y mírala dispararse.

PASO 1: ABRE EL CODESPACE

- Escanea el QR o usa el enlace de la guía TALLER.md.
- Inicia sesión con tu cuenta gratuita de GitHub y pulsa Create codespace.
- Espera 1 a 2 minutos. Se abre VS Code en el navegador, con una terminal abajo.

Abrir el Codespace

PASO 1: LEVANTA EL ENTORNO

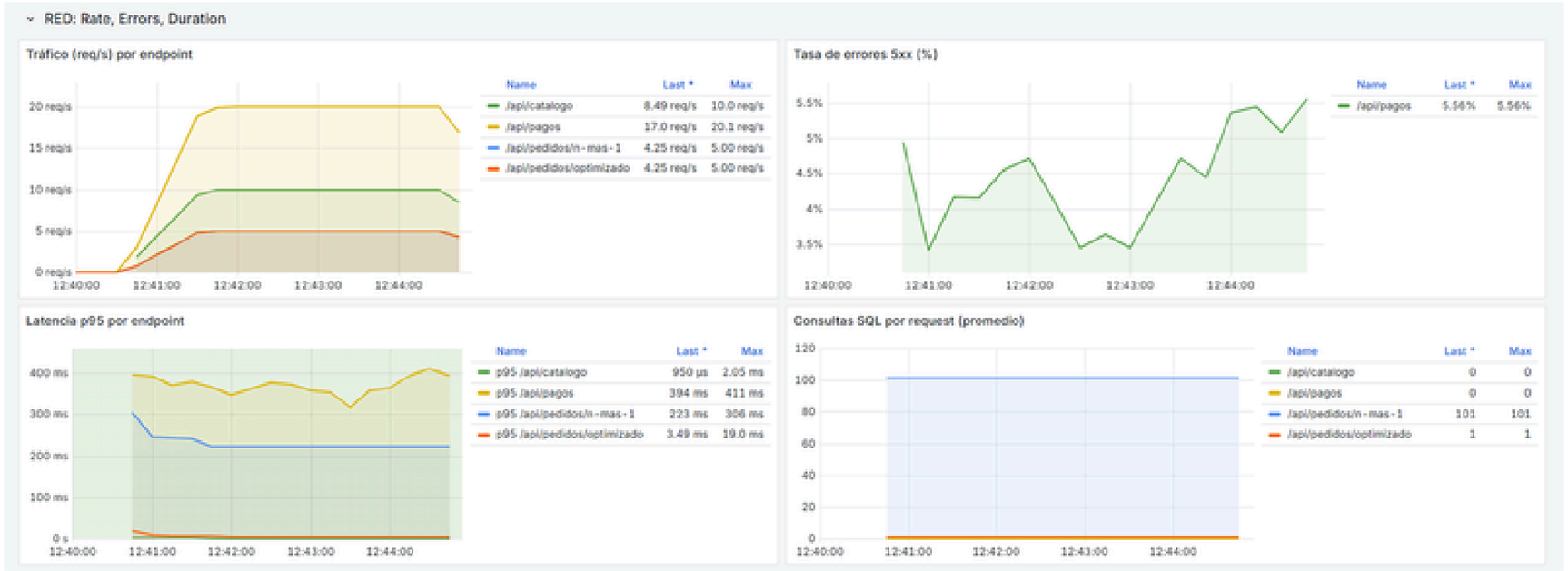
En la terminal ejecuta estos comandos. La primera vez tarda unos 2 minutos, y la respuesta debe decir status UP.

```
docker compose up -d --build

curl -s localhost:8080/actuator/health
{"status":"UP"}

Opción B, en tu computador con Docker Desktop, antes de lo anterior:
git clone -b claude/project-thread-ga3dib https://github.com/JuanTobon7/arquitectura-software.git
cd arquitectura-software/observabilidad-demo
```

PASO 1: ABRE GRAFANA



Pestaña PORTS, ícono del globo en el puerto 3000. No pide usuario ni contraseña. Sin tráfico estará casi vacío; con tráfico se verá así.

PASO 2: GENERA TRÁFICO

k6 simula usuarios en segundo plano: unas 40 peticiones por segundo durante 6 minutos. Espera 1 minuto y vuelve a Grafana.

```
DURACION=6m docker compose --profile carga run -d --rm k6
```

Qué recibe cada endpoint:
POST /api/pagos: 20 peticiones por segundo
GET /api/catalogo: 10 peticiones por segundo
GET /api/pedidos/n-mas-1: 5 peticiones por segundo
GET /api/pedidos/optimizado: 5 peticiones por segundo

MISIÓN 1: ¿EL PROMEDIO MIENTE?

En Grafana entra a Explore, elige Prometheus y cambia a modo Code. Pega cada consulta con Add query.

Promedio de /api/pagos:
sum(rate(http_server_requests_seconds_sum{uri="/api/pagos"}[1m]))
/ sum(rate(http_server_requests_seconds_count{uri="/api/pagos"}[1m]))

Percentil 95:
histogram_quantile(0.95, sum by (le) (rate(http_server_requests_seconds_bucket{uri="/api/pagos"}[1m])))

Percentil 99:
histogram_quantile(0.99, sum by (le) (rate(http_server_requests_seconds_bucket{uri="/api/pagos"}[1m])))

MISIÓN 1: RESPONDE

- ¿Cuánto da el promedio y cuánto el p99? ¿Cuántas veces mayor es el p99?
- Si tu objetivo fuera el 99% de los pagos en menos de 500 ms, ¿se cumple?
- Mira los hilos de Tomcat y la CPU en el dashboard. ¿El servidor está saturado? ¿Dónde está el problema?

MISIÓN 2: CAZA EL N+1

Los dos endpoints de pedidos devuelven lo mismo. Compruébalo en la terminal y luego busca la evidencia en Explore, con Loki.

En la terminal:

```
curl -s localhost:8080/api/pedidos/n-mas-1 | head -c 150; echo
curl -s localhost:8080/api/pedidos/optimizado | head -c 150; echo
```

En Explore, con Loki:

```
{app="api-demo"} | json | mdc_sql_consultas > 50
```

En el dashboard: paneles Latencia p95 por endpoint y Consultas SQL por request.

MISIÓN 2: RESPONDE

- ¿Cuál endpoint de pedidos es más lento, y por cuánto?
- ¿Cuántas consultas SQL hace cada uno? ¿Por qué ese número exacto, si hay 100 pedidos?
- Abre PedidoRepository y PedidoController. ¿Qué cambia entre findAll y findAllConItems?

MISIÓN 3: DE LA ALERTA AL LOG

Abre Prometheus desde la pestaña PORTS, puerto 9090, y entra al menú Alerts. Luego ve a Explore, con Loki.

Ver los errores:
{app="api-demo", level="ERROR"}

Contar errores por minuto, solo con logs:
sum(count_over_time({app="api-demo", level="ERROR"}[1m]))

En el dashboard: panel Tasa de errores 5xx.

MISIÓN 3: RESPONDE

- ¿Qué alerta está en estado FIRING, y para qué endpoint?
- ¿Qué porcentaje de peticiones falla? ¿Hay errores en otros endpoints?
- ¿Qué dice el mensaje de los logs? ¿El error es de nuestra API o de una dependencia?

RETO: TU PROPIA ALERTA DE P99

Agrega esta regla al final de infra/prometheus/alertas.yml, alineada con las otras. Recarga y mira Alerts: pasa de Pending a Firing en menos de un minuto.

```
- alert: LatenciaP99Alta
  expr: |
    histogram_quantile(0.99,
      sum by (le, uri) (rate(http_server_requests_seconds_bucket{uri!~"/actuator.*"}[2m]))
    ) > 1
  for: 30s
  labels:
    severity: warning

Para recargar Prometheus:
curl -X POST localhost:9090/-/reload
```

COMPARA TUS RESPUESTAS

Misión 1: promedio de 103 ms y p99 de 1.574 ms. La API no está saturada; la causa es la pasarela.

Misión 2: 101 consultas y p95 de 224 ms contra 1 consulta y 5 ms con JOIN FETCH.

Misión 3: TasaErroresAlta en /api/pagos, con 4,4% de errores por timeout de la pasarela.

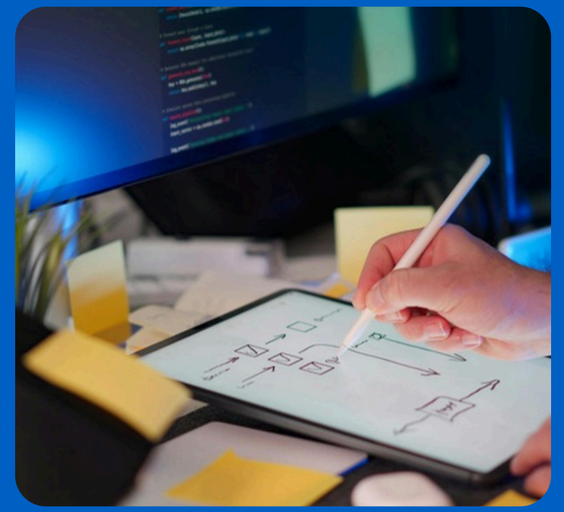




SI ALGO FALLA

- Connection refused: la API aún arranca. Espera 20 segundos y repite.
- Paneles vacíos o Loki sin líneas: revisa que lanzaste k6 y espera un minuto.
- La alerta no aparece: revisa la indentación del archivo YAML.





¿PREGUNTAS?

Al terminar: docker compose down y detén tu Codespace